

Agentic dMRV Data Validation

Agentic Data Validation PoC for pre-VVB data quality checks

PRIMARY GOAL: VERIFICATION READINESS

A Validation Agent authors executable rules; a Capability Extension Agent handles unsupported capabilities; a deterministic engine validates the filtered dataset.

1 Write a validation rule

“Planting date must be within the season.”

2 Validation Agent composes operators

Normal rule path uses approved operators only.

3 Engine runs locally

CSV + DuckDB/Python/GeoPandas.

4 Review field-season findings

PASS • FAIL • REVIEW • N/A

Filtered validation data stays inside the validation environment.

Why build this?

The current challenge is not data collection — it is making the collected evidence consistent, explainable, and verification-ready.

Many data sources

Farmer diaries, geotagged photos, RicelD, contracts, field geometry, season definitions, and reference datasets must agree.

Rules evolve

Validation requirements change by methodology, project, season, or VVB feedback. Hard-coding every change creates engineering bottlenecks.

Manual review does not scale

Thousands of field-season records make manual cross-checking slow and error-prone before verification.

PoC objective

Create a flexible validation layer that checks every lot/field + season before VVB submission — while allowing new rules to be defined without a normal engineering deployment cycle.

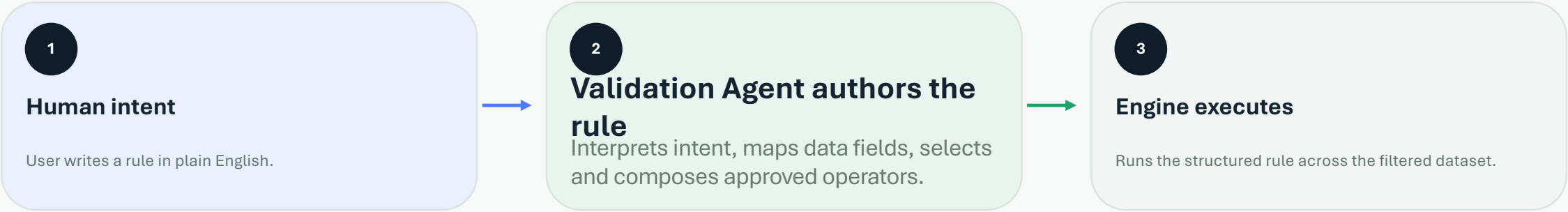
Verification readiness

Rule flexibility

Safe AI use

Two bounded agentic roles, deterministic execution

The Validation Agent composes rules. When a capability is missing, the Capability Extension Agent helps developers extend the operator library. Field-level validation remains deterministic.



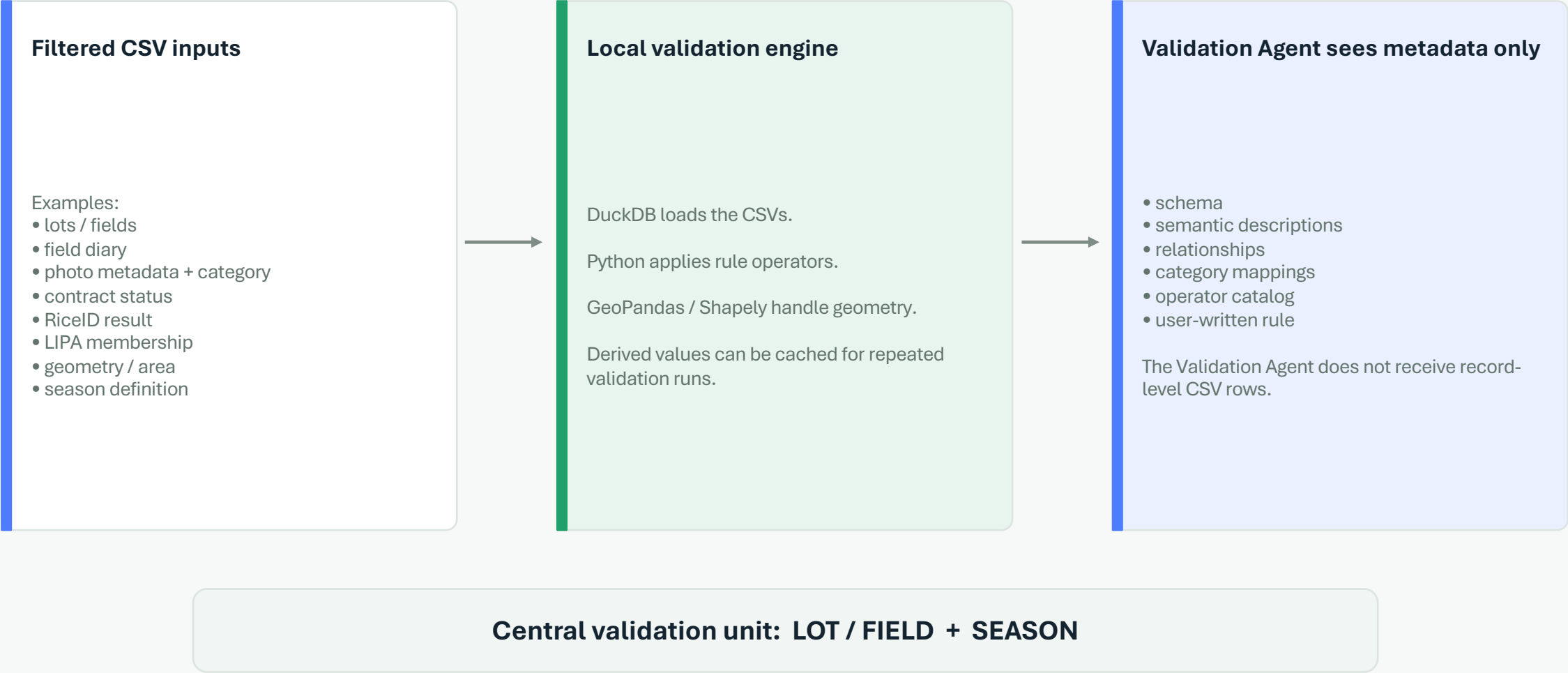
Why this matters

- ✓ Deterministic, reproducible validation results
- ✓ Auditable rules that can be versioned
- ✓ No agent call per field-season
- ✓ Fast batch execution at scale

Agentic where flexibility is valuable. Deterministic where auditability matters.

Filtered validation dataset

For the PoC, the system reads purpose-built CSV exports containing only validation-relevant information.



Initial C1–C11 rule pack

The first eleven checks already cover most of the operator families needed for a reusable validation language.

Temporal

- C1 Planting after straw management
- C4 Planting within season
- C6 Photo dates within season

Photo sequence / conditional

- C2 First photo = Installation or Non-rice
- C5 Non-rice should not have other photos

Cross-source consistency

- C3 Diary crop ↔ photo categories
- C7 Diary crop ↔ RiceID

Completeness + spatial / numeric

- C8 Lot has GeoJSON
- C9 Declared area ↔ GeoJSON area

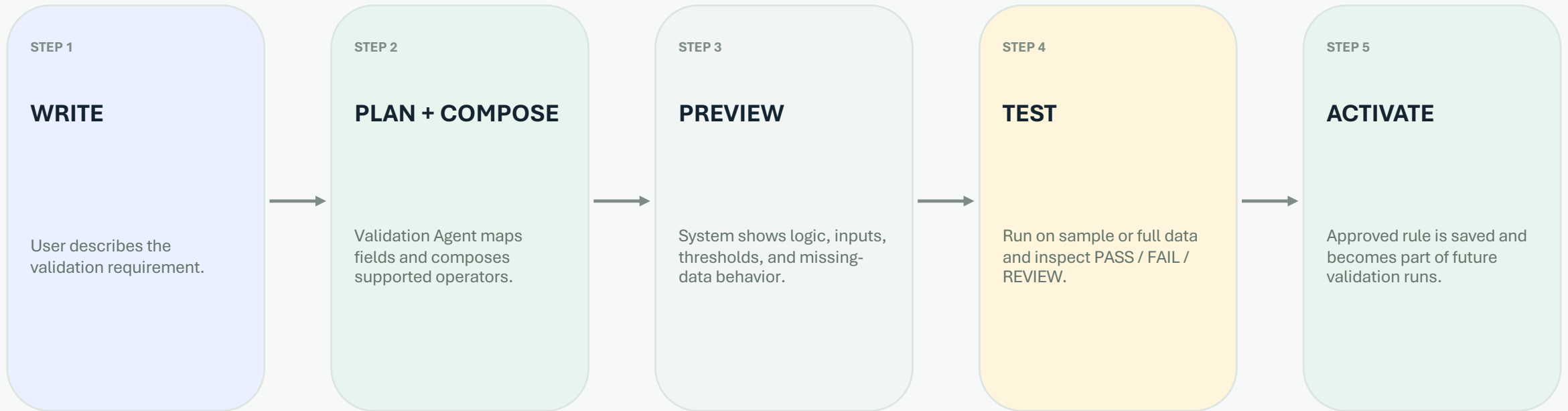
Relational + membership

- C10 Farmer has signed contract
- C11 Lot is in LIPA 2025 DS26

Design implication: implement reusable primitives first — not eleven one-off functions.

Agentic rule authoring workflow

A new rule should move from plain language to an executable check without a normal code deployment.



Most new rules require no code deployment: the Validation Agent composes existing operators into a structured rule.

The operator library is the reusable foundation

The Validation Agent can create many new rules by composing a small set of trusted primitives.

Temporal

before • after • between • days_between

Categorical

equals • in • mapped_equals

Sequence

first • last • count • filter

Logic

IF • AND • OR • NOT

Spatial / numeric

geometry_area • within • distance • % difference

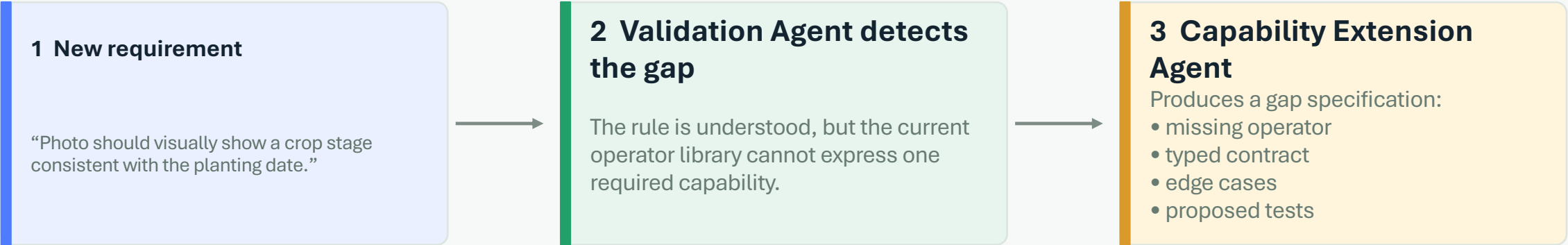
Relational / document

exists • related_record_exists • signed • membership

One new operator can unlock many future rules — so capability grows faster than code size.

When a new rule is too complex

Unsupported rules trigger a second bounded agentic workflow: propose the missing capability, generate a reviewed implementation draft, then register it only after developer approval.



Generated code never enters the active operator registry without developer approval.

Three challenge rules for capability-gap testing

Keep these intentionally unsupported by the initial operator library. They are used later to prove that the system can detect, extend, and close capability gaps.

1 Visual crop-stage evidence VISION

Rule to enter:

“A planting photo should visually show a field condition consistent with planting or early crop establishment.”

Why it should fail initially:

Metadata operators cannot interpret image content.

Expected missing capability:
`classify_field_stage(photo)`

2 Representative photo location SPATIAL

Rule to enter:

“A geotagged photo must be inside the field and reasonably representative, not captured too close to the field boundary.”

Why it should fail initially:

`within_polygon()` is not enough.

Expected missing capability:
`distance_to_boundary()`

3 Plausible crop-stage progression SEMANTIC

Rule to enter:

“Crop observations must follow a plausible seasonal progression and must not move backwards, e.g. Harvest → Vegetative.”

Why it should fail initially:

Sequence operators do not understand ordered crop stages.

Expected missing capability:
`validate_stage_progression()`

Capability-gap test

Enter rule → expect `UNSUPPORTED_CAPABILITY` → generate operator/test draft → developer reviews → register operator → retry the exact same rule → expect `COMPILED`.

Streamlit PoC experience

The product should feel like a verification-readiness control center — not a generic chatbot.

dMRV Validation

Dashboard

Data

Rules

Field Review

Agentic Validation Rule Builder

Describe the validation requirement

Farmer declared area should not differ by more than 10% from the GeoJSON area.

Ask Agent

Agent plan

geometry_area()

% difference ≤ 10%

missing → REVIEW

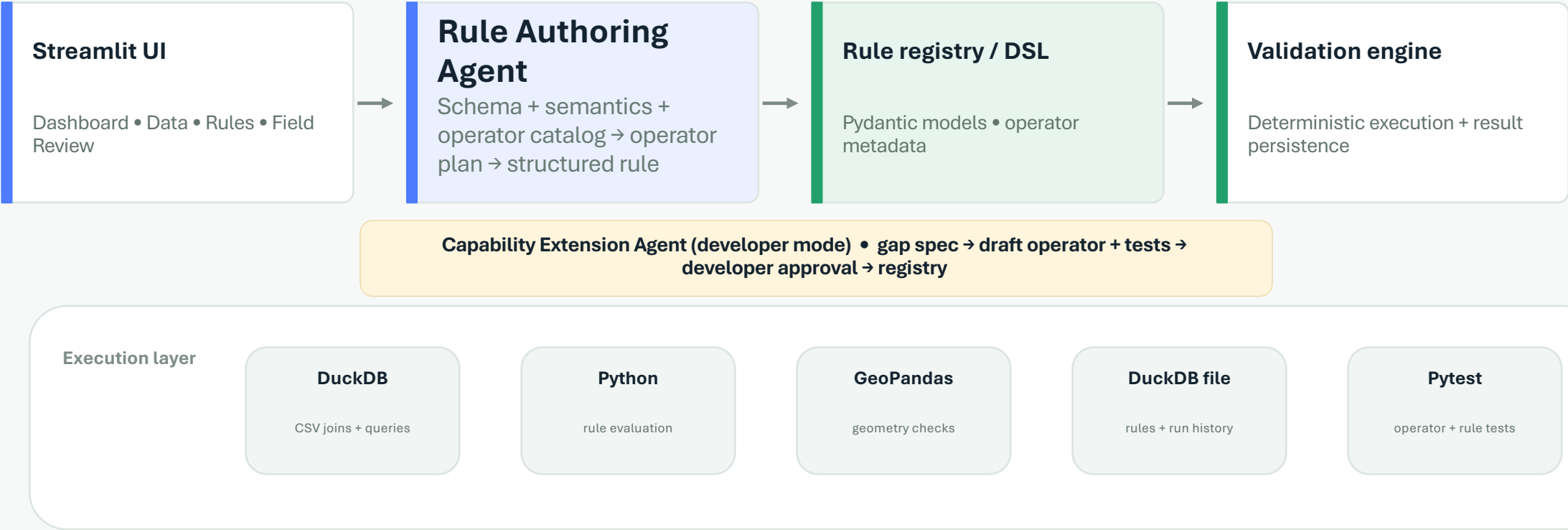
Preview

Test Rule

Four pages are enough for v1: Dashboard • Data • Rules • Field Review

PoC technical architecture

Two bounded agentic roles can use the same LLM runtime. No MCP or multi-agent framework is required for the PoC.



Suggested stack: Python 3.11+ • Streamlit • DuckDB • Pydantic • Pandas/Polars • GeoPandas/Shapely • YAML • pytest

Build sequence with Claude Code

Build the deterministic core first. Add the Rule Authoring Agent, then the developer-gated Capability Extension Agent.

1

Canonical data model

Define lot/field + season entities and CSV adapters

2

Operator framework

Registry, types, execution contract, logical composition

3

C1–C11 rules

Implement and test initial rule pack

4

Synthetic dataset

Seed known failures and expected findings

5

Validation engine

Batch execution + PASS/FAIL/REVIEW results

6

Streamlit UI

Dashboard, Data, Rules, Field Review

7

Validation Agent

Natural language → operator plan → validated DSL

8

Rule lifecycle

Preview → Test → Approve → Activate

9

Capability Extension Agent

Gap spec → code/test draft → developer review → register

10

Real CSV

Map filtered export into canonical model

Target: demonstrable by Day 3 • strong internal PoC in ~5–10 working days

What success looks like

The PoC should prove the business value first — not the sophistication of the AI stack.

1 Verification readiness

Run all active checks across the filtered validation dataset and clearly identify which field-seasons are ready, failed, or need review.

Primary success metric: fewer unresolved data-quality issues before VVB submission.

2 Agentic rule flexibility

Enter unseen business rules in natural language and compile most of them using approved operators without changing application code.

Measure: percentage of unseen rules compiled successfully.

3 Two bounded agentic roles

Show that the Rule Authoring Agent composes executable rules and the Capability Extension Agent closes unsupported gaps through developer-reviewed code generation.

No record-by-record agent validation is required.

PoC success test: C1–C11 work deterministically, unseen rules compile from existing operators, and the 3 challenge rules correctly trigger then close capability gaps.